

1. Explain briefly different database languages and their significance in DBMS?

- Data Definition Language (DDL): is used to define and describe the structure of a database, including the creation, modification, and deletion of database objects like tables, indexes, and schemas.

Operations:

- o Create Database Objects like the structure of tables, indexes, and other objects
`CREATE TABLE Students (ID INT, Name VARCHAR(50), Age INT);`
- o Alter Database Objects: Modifies the structure of an existing object
`ALTER TABLE Students ADD Gender VARCHAR(1);`
- o Drop Database Objects: Deletes a database object
`DROP TABLE Students;`
- DDL specifies data storage methods and access techniques, ensuring efficient data management while hiding technical details from the user.
- Data Manipulation Language (DML): is used to work with data that is already present in a database.
- It allows users to access, insert, delete, and modify data.

Operations:

- o Retrieve data stored in the database using commands like SELECT
`SELECT * FROM Students`
- o Insert data into a table
`INSERT INTO Students VALUES (1, 'John', 21);`
- o Delete data from a table
`DELETE FROM Students WHERE ID = 1;`
- DML enables users to directly interact with data, allowing them to make specific requests for information, insert new data, delete unnecessary data, and modify existing data as needed.
- Consistency Constraints: These are rules that ensure the database remains accurate and reliable. The most common types include:
 - o Domain Constraints: Each attribute must contain values from a specific domain (e.g., a PhoneNumber field must only contain numeric values).
 - o Referential Integrity: Ensures that data in one table corresponds with data in another. For instance, a CustomerID in an Orders table must match an existing CustomerID in the Customers table.
- Assertions: These are conditions that must always be true in the database. For example, an assertion could enforce that an employee's salary is always greater than zero
 - Authorization

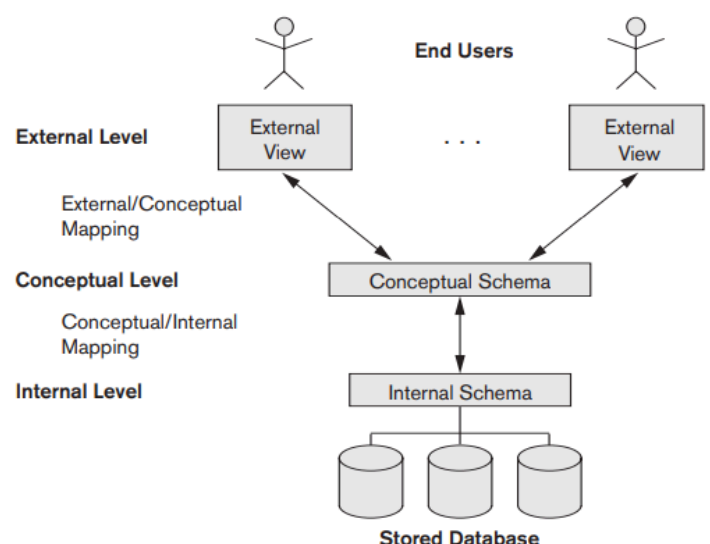
We may want to differentiate among the users as far as the type of access they are permitted on various data values in the database. These differentiations are expressed in terms of authorization, the most common being: read authorization, which allows reading, but not modification, of data; insert authorization, which allows insertion of new data, but not modification of existing data; update authorization, which allows modification, but not deletion, of data; and delete authorization, which allows deletion of data

2. State importance of each of these languages

- Data Definition Language (DDL)
 - o Defines Database Structure: DDL provides the necessary commands to create and modify the structure of a database, such as tables, indexes, and relationships.
 - o Organises Data: It organises data storage by specifying how data should be stored, accessed, and maintained, ensuring efficient database performance.
 - o Enforces Data Integrity: DDL defines constraints like primary keys, foreign keys, and check constraints, ensuring the accuracy and consistency of the data.
 - o Abstracts Technical Details: By defining the schema, DDL hides complex storage details from the user, allowing them to focus on higher-level tasks.
- Data Manipulation Language (DML)
 - o Facilitates Data Access: DML allows users to retrieve, insert, delete, and update data, making it easier to manipulate the stored information.
 - o Enables Data Querying: Declarative DML enables users to interact with the data without worrying about how it's accessed or retrieved, making it user-friendly for non-technical users.
 - o Allows Data Modifications: Through procedural or non-procedural methods, DML helps in efficiently modifying large datasets for analysis, reporting, or maintenance purposes.
 - o Supports Transactional Operations: DML commands are used to manage day-to-day operations in the database, supporting the basic CRUD (Create, Read, Update, Delete) operations.

3. Explain database system architecture with neat diagram

- The main aim of three schema architectures is data independence which means users should not be able to interact with data directly. Users should not be aware about the fact exactly data is stored.
- View Level: is how to represent data to users. For example: A student is trying to login to his college website using credentials. Once they login then they will be able to see the data. This is known as the student view. This view will not be the same for all users. UI designers work at this level.
- Conceptual schema: is the way in which data is represented i.e. the structure of data. For example: if we have student data then we have columns in student data (RollNo, Name, Address). Database designers work at this level
- Physical Schema: specifies at which location data is stored. Administrators decide at which location to store which data. Database administrator works at physical level



4. Explain in brief following types of NoSQL databases

- **Key-Value Pair:** Designed for storing data in schema-less way. Data is stored as key-value pairs where each key is unique, and its corresponding value can be any type of data(eg: strings, numbers and even more complex objects). It is best for Fast data retrieval, caching, session storage, and simple data lookups. These types of databases are used for Storing user sessions in web applications.
- **Example:** Cassandra, DynamoDB, Azure Table Storage etc
- **Graph based:** These databases are designed to store and manage data that is highly interconnected. Data is stored as nodes and edges, which represent entities and their relationships. It is best for Managing complex relationships, like social networks, recommendation engines, and fraud detection. It is used in Social media platforms where users, their connections, and interactions are analysed.
- **Example:** Neo4j, Amazon Neptune
- **Column Oriented Graph:** A wide-column database is a type of NoSQL database in which the names and format of the columns can vary across rows, even within the same table. Wide-column databases are also known as column family databases. A column is only written if there's a data element for it. Each data element can be referenced by the row key, but querying for a value is optimised like querying an index in a RDBMS, rather than a slow table scan.
- **Document Oriented:** These databases store data as documents (JSON). Each document can have a flexible schema, allowing for varying structures of data within the same collection. It is best for semi-structured data when flexibility is needed for the data model. It is widely used in product catalogues in e-commerce platforms where each product might have different attributes

5. What are the advantages of NoSQL databases?

- **Scalability:** NoSQL databases are designed to scale horizontally across many servers, meaning they can handle massive amounts of data by adding more servers rather than upgrading a single powerful machine.
- This makes it more cost-effective to handle growing datasets
- **Flexibility in Schema:** NoSQL databases have flexible, schema-less structure, meaning you don't need to define the structure of your data upfront.
- It is ideal for unstructured / semi-structured data like JSON / XML, where the format of data is not fixed
- **High Availability and Fault Tolerance:** NoSQL databases provide support for distributed systems, making them more resilient to failures.
- Data is often replicated across multiple nodes, ensuring that if one node fails, the data remains available.
- **Better Performance:** NoSQL databases are optimised for operations like fast reads and writes
- This results in faster performance for applications that require high-throughput operations, such as real-time data analytics, caching, and content delivery.
- **Handling Large Volumes of Data:** NoSQL databases are well-suited for handling large volumes of content
- This makes them ideal for big data applications like IoT, social networking, and recommendation systems.

- Distributed Architecture: NoSQL databases are designed to distribute data across multiple servers or data centres.
- This improved reliability, and the ability to handle geographically distributed users with lower latency.
- Cost-Effectiveness: NoSQL databases can run on clusters of inexpensive commodity hardware instead of relying on expensive, high-performance machines.
- This reduces the overall cost of infrastructure, especially as the system scales.

**6. Differentiate between SQL and NoSQL. How to decide which is relevant for project development?
How to decide whether SQL or NoSQL is relevant for project development**

No	SQL	NoSQL
1	Primarily called as relational databases	Primarily called as distributed database
2	Databases are table-based	Databases are document based, key-value pairs, graph based
3	Databases have predefined and fixed schema	Databases have dynamic schema for unstructured data.
4	Databases are vertically scalable	NoSQL databases are horizontally scalable.
5	Databases uses SQL for defining and manipulating the data	Database queries are focused on collection of documents
6	Database examples: MySQL, Sqlite, MS-SQL	Database examples: MongoDB, Redis, Cassandra
7	Databases are good fit for complex queries	Databases are not a good fit for complex queries.
8	Databases are not best fit for hierarchical data storage.	Database fits better for the hierarchical data storage as it follows the key-value pair way of storing JSON data
9	Databases are best fit for transactional type applications, as it is more stable	Databases are not comparable and stable enough in complex transactional applications.
10	Uses ACID Properties	Uses CAP theorem

Choose SQL if your project needs:

- Structured, relational data with well-defined schemas.
- Complex queries and transactional support.

Choose NoSQL if your project requires:

- Handling large volumes of unstructured or semi-structured data.
- Flexibility in schema and fast read / write, with lesser consistency.

7. Explain different phases of data analytics lifecycle?

- Phase 1: Discovery: Understand the problem and resources available
- They also check if the business has tried similar projects before and can learn from them
- The team looks at the resources (like people, technology, time, data) they have
- Then frame the problem in a way that data analytics can help solve
- They also come up with initial guesses (called hypotheses) about what the data might reveal

- Phase 2: Data Preparation: Make the data clean and accessible for analysis
- The team creates a sandbox where they can work with data.
- They then bring in the data (ETL), clean it, and organise it so it's ready for analysis.

- Phase 3: Model Planning: Plan the methods and models to be used
- The team studies the data to see how different variables relate to each other
- They then choose the best variables and models to use for analysis in the next phase

- Phase 4: Model Building: Build and test models to analyse the data
- The team prepares the data by splitting it into different sets (training and testing sets).
- They then build the models and test if their existing tools can handle it or if they need better tools

- Phase 5: Communicate Results: Share key insights and results with stakeholders
- The team checks if their analysis solved the problem based on the success criteria from Phase 1.
- They identify key findings and create a report to share in a simple way.

- Phase 6: Operationalize: Implement the solution and monitor its performance
- The team delivers the final reports, code, and documentation, and sometimes runs a small pilot project to see how the solution works in real-world settings.
- This phase ensures the results are applied and monitored in the business.

1. Explain R Nuts and bolts in detail.

- R is a free and open-source software, allowing anyone to use it as a programming language and environment developed for statistical analysis.
- The graphical capabilities of R provide a fully programmable graphics that are better than most other statistical and graphical packages.
- R has no licence restrictions other than ensuring our freedom to use it at our own discretion
- Anyone can provide new packages, and the quality packages available for R helps in software development and sharing.
- R has over 4800 packages available from multiple repositories specialising in topics like econometrics, data mining, spatial analysis, and bioinformatics.
- R is cross-platform. R runs on many operating systems and different hardware.

- R plays well with many other tools, importing data, for example, from CSV files directly from Microsoft Excel, MySQL, and SQLite.
- It can also produce graphics output in PDF, JPG, PNG, and SVG formats, and table output for LATEX and HTML.

2. What are different data types supported in R. Explain them in brief

- Numeric data types store numbers, which can be either integers or floating-point numbers.
- Used for mathematical calculations, statistical analysis, and operations involving numbers.

Example:

R

```
num <- 42
pi_value <- 3.14
print(num)
print(pi_value)
```

- Integers are whole numbers without decimal points. In R, integers are explicitly defined using the `as.integer()` function
- Used when precise integer values are required

Example:

R

```
int_val <- as.integer(5)
print(int_val)
```

- `Character(String)`: type is used to store text strings, which can include letters, words, or sentences.
- Useful for storing and processing text data, labels, and categorical values.

Example:

R

```
char_val <- "R programming"
print(char_val)
```

- Logical: data types store Boolean values, which are either TRUE or FALSE.
- These are often the result of logical conditions or comparisons used in control flow, logical operations, or data filtering

Example:

```
R

logical_val <- TRUE
print(logical_val)
```

- Complex numbers: have a real and an imaginary part.
- They are written in the form $a + bi$, where i is the imaginary unit.
- Useful for advanced mathematical operations that require imaginary numbers

Example:

```
R

complex_num <- 2 + 3i
print(complex_num)
```

3. Explain different ways of reading data in R

- `read.table()` and `read.csv()`: are commonly used for reading tabular data.
 - o `read.table(file, header=TRUE, sep="\t")` – Reads a tabular text file.
 - o `read.csv(file, header=TRUE)` – Reads a CSV file.
- `readLines()`: reads lines from a text file, making it useful for non-tabular / unstructured data
 - o `readLines(file)` – Reads each line as a character string and returns a vector of lines.
- `source()`: is used to read and execute R code from an external file. It is typically used for running scripts.
 - o `source(file)` – Loads and executes the R code in the file.

`dget()`: reads R objects that were saved in a text format using `dput()`, used for restoring individual R objects

- o `dget(file)` – Reads and reconstructs an R object from a text file

`load()`: is used to read data saved in an R workspace or `.RData` file. It restores all saved objects.

- o `load(file)` – Loads the workspace and makes the objects available in the R environment.

`unserialize()`: reads and reconstructs a single R object saved in binary form, used in more advanced apps

- o `unserialize(file)` – Recreates an R object from its binary representation.

4. Name and Explain the I) Basic Operations and II) Basic statistics pertaining to R

I. Basic Operations in R

- Arithmetic operations: These include addition, Subtraction, Multiplication, Division and exponential

```
> a <- 10
> b <- 5

> sum <- a + b
> sum
[1] 15

> product <- a * b
> product
[1] 50

> Division<-a/b
> Division
[1] 2

> Expo<-x^y
> Expo
[1] 25
```

- Relational Operations: used for comparison and include greater than, less than, equal to, not equal to

```
> a=10
> b=4
>
> a>b
[1] TRUE
> a==b
[1] FALSE
> a!=b
[1] TRUE
```

- Logical operations: include AND, OR, NOT

```
a=10
b=4

> (a > 5) & (b < 10) # TRUE
[1] TRUE
> (a == b) | (b > 3) # TRUE
[1] TRUE
```

- Assignment operations: used to assign values to the variable

```
> x<-25 #
> x
[1] 25
```

- Indexing and slicing:

```
> vec <- c(1, 2, 3, 4, 5)
> vec[3] # Accesses the third element, which is 3
[1] 3
```

II. Basic statistics pertaining to R

- Mean: is the average value of a data set.

```
> data <- c(10, 20, 30, 40, 50)
> mean(data) # Returns 30
[1] 30
```

- Median: is the middle value of a data set

```
data <- c(10, 20, 30, 40, 50)
> median(data)
[1] 30
```

- Mode: R doesn't have a built-in function for mode, but it can be calculated using custom code.

- Variance: measures the spread of data. It is calculated using the var() function. Example:

```
data <- c(10, 20, 30, 40, 50)
> var(data)
[1] 250
```

- Standard Deviation: is the square root of the variance

```
data <- c(10, 20, 30, 40, 50)
> sd(data)
[1] 15.81139
```

- Summary Statistics: a function, which gives an overview of the basic statistics for data

```
data <- c(10, 20, 30, 40, 50)
> summary(data)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
    10     20     30     30     40     50 
```

- Correlation and Covariance can calculate the relation between two variables

```
> x <- c(1, 2, 3, 4, 5)
> y <- c(2, 4, 6, 8, 10)
> cor(x, y) # Returns 1 (perfect correlation)
[1] 1
> cov(x, y) # Returns covariance
[1] 5
```

4. What are factors in R-language? How are they created? How are they modified? Explain with R code.

- Factors are data structures used to represent categorical data.
- They store both the values and the categories.
- They are useful for statistical modelling where categorical variables are treated as distinct levels, rather than numerical values.

Key Characteristics of Factors:

- Factors are stored as integers, with each unique level being assigned an integer.
- They have a levels attribute, which contains all the unique values or categories.
- They can be ordered or unordered. Ordered factors are useful when there is a natural ranking of categories (e.g "Low", "Medium", "High").

Creating Factors in R:

```
gender <- factor(c("Male", "Female", "Female", "Male", "Male"))  
print(gender)
```

```
[1] Male   Female Female Male   Male  
Levels: Female Male
```

Specifying Levels Manually:

```
education <- factor(c("Bachelor", "Master", "PhD", "Bachelor"), levels = c("Bachelor", "Master", "PhD"))  
print(education)
```

Modifying Factors in R:

```
levels(gender) <- c("F", "M")
```

- The levels of gender are renamed from "Female" and "Male" to "F" and "M", respectively.

Adding / Dropping Levels:

```
# Adding a level to a factor  
levels(education) <- c(levels(education), "Associate")  
  
# Dropping unused levels  
education <- factor(education, levels = c("Bachelor", "Master"))  
print(education)
```